Proxy Server with Load Balancer — Interview Preparation Guide

This document summarizes your proxy server and load balancer project — explaining its technical depth, interview potential, architecture, and improvements. It also includes a ready-to-use interview pitch, diagram explanation, and GitHub README overview.

1. Project Overview

1. Multi-threaded HTTP Proxy Server implemented in C.
2. Implements in-memory caching using an LRU-based linked list.
3. Thread synchronization using semaphores and mutex locks.
4. Round-robin Load Balancer to distribute clients across multiple proxy instances.
5. Implements socket programming concepts — socket(), bind(), listen(), accept(), connect(), send(), recv().

2. Why This Project Is Strong

1. Demonstrates strong grasp of OS, networking, and systems programming.
2. Integrates concurrency (threads, synchronization) with real networking logic.
3. Reflects production-level design concepts — similar to Nginx or HAProxy.
4. Clear separation of responsibilities: Load balancer handles routing, proxies handle caching and HTTP parsing.
5. Practical and scalable: supports multiple clients, handles connection closing, caching, and request forwarding.

3. System Architecture

The architecture consists of three major components: 1. **Load Balancer:** Accepts incoming client connections and forwards them to proxy servers using a round-robin policy. 2. **Proxy Server:** Handles client HTTP requests, forwards them to remote servers, caches responses, and returns cached results when available. 3. **Cache:** In-memory linked list structure storing recently accessed URLs with LRU eviction.

4. Interview Pitch (2–3 Minutes)

“I implemented a multi-threaded HTTP proxy server with an integrated caching mechanism and a round-robin load balancer. The proxy server accepts client requests, parses HTTP headers, and either serves data from the cache or fetches it from the remote server. To improve scalability, I added a round-robin load balancer that distributes client connections evenly across multiple proxy instances. This project helped me deepen my understanding of socket programming, synchronization, and performance optimization.”

5. Key Interview Talking Points

1. Cache replacement policy: LRU-based linked list.
2. Concurrency control: mutex for cache updates, semaphore for client count.
3. Round-robin algorithm ensures fair load distribution.

4 Handles HTTP/1.0 and HTTP/1.1 requests.
5 Memory management: uses malloc/free carefully to prevent leaks.
6 Scalable up to hundreds of concurrent connections.

6. Future Improvements

1 Add health checks for proxy nodes.
2 Implement detailed logging and performance metrics.
3 Add HTTPS (TLS passthrough or termination using OpenSSL).
4 Containerize using Docker and enable dynamic proxy registration.
5 Implement least-connections or weighted round-robin strategies.

7. GitHub README Template

````
# Proxy Server + Load Balancer
A multithreaded HTTP proxy server in C with caching and a round-robin load balancer.

## Features
- Multi-client handling via threads
- Caching with LRU eviction
- Round-robin load balancing
- Graceful socket and memory handling

## Build & Run
```bash
gcc proxy_server.c -o proxy -lpthread
gcc load_balancer.c -o balancer -lpthread
./proxy
./balancer ...
```

## Future Work
- Health checks, HTTPS support, containerization
````

8. Final Verdict

This project is a top-tier system-level implementation for interviews. It demonstrates your ability to build real networked systems, handle concurrency, and optimize for performance. With minor additions (Docker, logging, health checks), it becomes production-grade.